

Internship Task

Week 2

**Virtualization, Containerization &
Python Scripting**

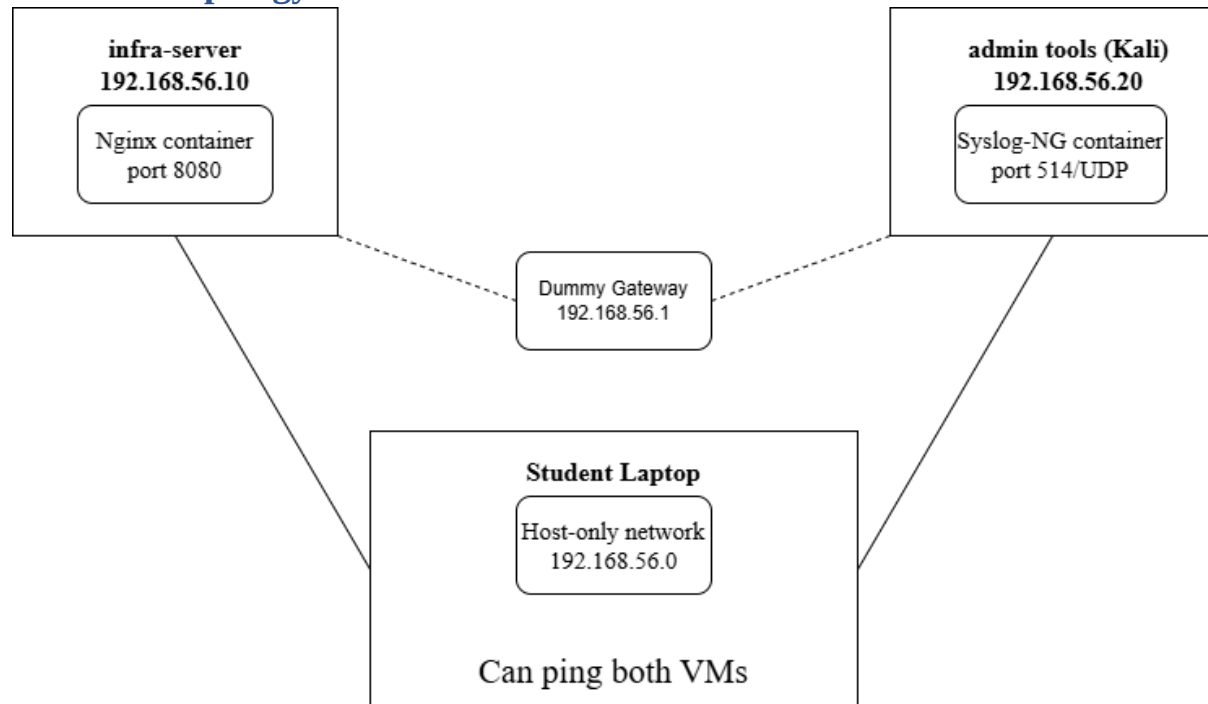
Submitted by: Rania Muskan Malik

Date: 7/20/2026 1:30:22 AM

Table of Contents

Network Topology	2
IP/Port Table	2
Task 1: Virtualization	2
Task 2: Containerization	4
Task 3: Python Automation	5
set_ip.py: takes --ip, --gw, --iface; applies via Netplan; logs to /var/log/ip-config.log	5
monitor.py	5
Bonus: Packet Analysis Section	6
1. HTTP traffic (admin-tools → Nginx) – screenshot + header analysis	6
2. ICMP echo between VMs – prove connectivity & TTL values	7
3. Syslog UDP 514 (infra-server → Syslog-NG) – show log payload	9
4. Security note: plaintext HTTP vs HTTPS	9
5. Extra: capture with tcpdump -i port 514 -w syslog.pcap and open in Wireshark	10
Reflection: What I Learned This Week	10

Network Topology



IP/Port Table

Device	IP Address	Port	Purpose
infra-server	192.168.56.10	8080 (TCP)	Nginx container
admin-tools (Kali)	192.168.56.20	514 (UDP)	Syslog-NG container
Dummy Gateway	192.168.56.1	-	Not routable
Host	192.168.56.0	-	

Task 1: Virtualization

Ubuntu 22.04 was built in Virtual Box and an existing Kali VM was used. On both of them Adapter 1 from the network settings was set to host-only and a static ip address was applied.

```

rania@infra-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:6f:d7:04 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe6f:d704/64 scope link
        valid_lft forever preferred_lft forever

```

(infra-server showing a static ip of 192.168.56.10)

```

(rania@kali) ~/Desktop
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:57:c9:dc brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.20/24 brd 192.168.56.255 scope global noprefixroute eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe57:c9dc/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 2e:ff:53:02:bf:33 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever

```

(kali showing a static ip of 192.168.56.20)

```

Session Actions Edit View Help
(rania@kali) ~/Desktop
$ ping 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data:
64 bytes from 192.168.56.10: icmp_seq=1 ttl=64 time=1.58 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=64 time=7.41 ms
64 bytes from 192.168.56.10: icmp_seq=3 ttl=64 time=10.3 ms
64 bytes from 192.168.56.10: icmp_seq=4 ttl=64 time=3.57 ms
64 bytes from 192.168.56.10: icmp_seq=5 ttl=64 time=3.85 ms
64 bytes from 192.168.56.10: icmp_seq=6 ttl=64 time=35.4 ms
64 bytes from 192.168.56.10: icmp_seq=7 ttl=64 time=4.52 ms
64 bytes from 192.168.56.10: icmp_seq=8 ttl=64 time=9.11 ms
64 bytes from 192.168.56.10: icmp_seq=9 ttl=64 time=5.66 ms
64 bytes from 192.168.56.10: icmp_seq=10 ttl=64 time=6.42 ms
64 bytes from 192.168.56.10: icmp_seq=11 ttl=64 time=6.46 ms
64 bytes from 192.168.56.10: icmp_seq=12 ttl=64 time=16.4 ms
64 bytes from 192.168.56.10: icmp_seq=13 ttl=64 time=6.34 ms
64 bytes from 192.168.56.10: icmp_seq=14 ttl=64 time=4.58 ms
64 bytes from 192.168.56.10: icmp_seq=15 ttl=64 time=3.29 ms
64 bytes from 192.168.56.10: icmp_seq=16 ttl=64 time=4.49 ms
64 bytes from 192.168.56.10: icmp_seq=17 ttl=64 time=4.68 ms
64 bytes from 192.168.56.10: icmp_seq=18 ttl=64 time=2.29 ms
64 bytes from 192.168.56.10: icmp_seq=19 ttl=64 time=2.67 ms
64 bytes from 192.168.56.10: icmp_seq=20 ttl=64 time=3.88 ms
64 bytes from 192.168.56.10: icmp_seq=21 ttl=64 time=1.03 ms
64 bytes from 192.168.56.10: icmp_seq=22 ttl=64 time=2.29 ms
64 bytes from 192.168.56.10: icmp_seq=23 ttl=64 time=2.16 ms
64 bytes from 192.168.56.10: icmp_seq=24 ttl=64 time=3.79 ms
64 bytes from 192.168.56.10: icmp_seq=25 ttl=64 time=42.3 ms

```

(from the kali vm a successful ping to the infra-server)

Task 2: Containerization

Docker was installed on both VMs. infra-server runs an Nginx container on a bridge network (web-net), exposed on host port 8080. admin-tools runs a Syslog-NG container on a separate bridge network (log-net), exposed on UDP port 514.

```

ranial@infra-server:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
e381763db21d   nginx    "/docker-entrypoint..." 17 seconds ago Up 16 seconds 0.0.0.0:8080->8080/tcp
ranial@infra-server:~$

```

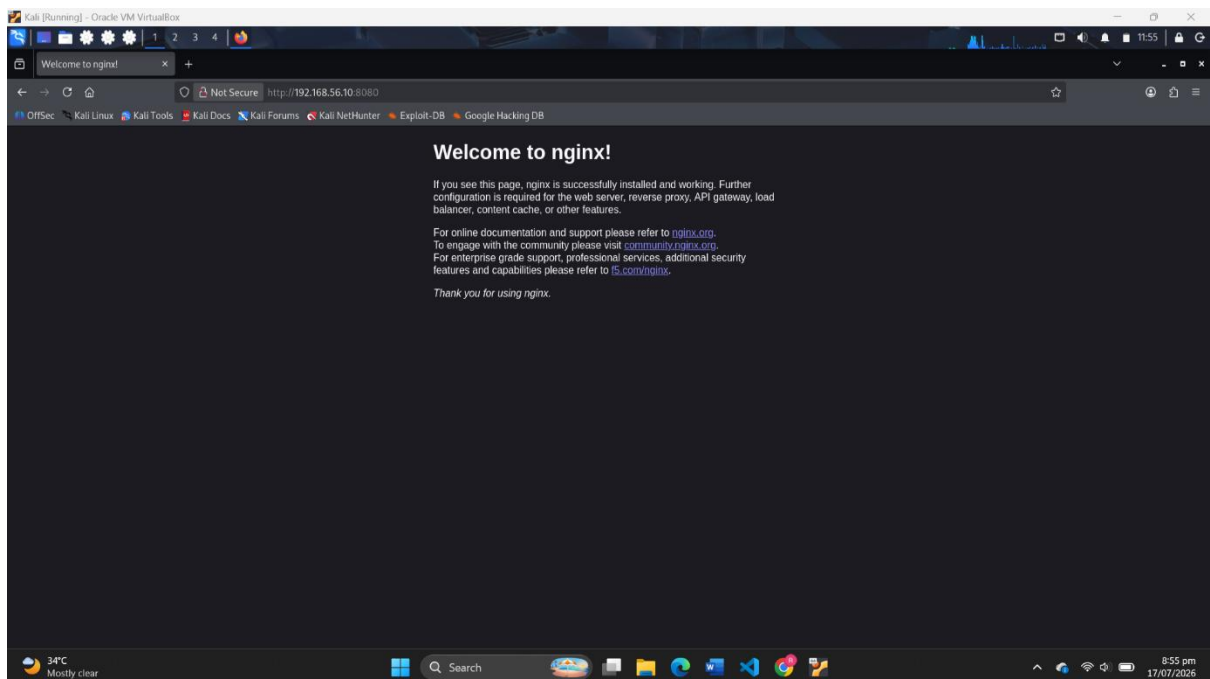
(docker ps on infra-server)

```

(rania@kali) - [~/Desktop]
$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
31926ec413ed   balabit/syslog-ng    "/usr/local/bin/entr..." 14 seconds ago Up 10 seconds (healthy) 601/tcp, 0.0.0.0:514->514/udp, [::]:514->514/udp, 6514/tcp    logger
(rania@kali) - [~/Desktop]
$

```

(docker ps on kali)



http://192.168.56.10:8080 (from admin-tools)

```

Session Actions Edit View Help
(rania@kali) - [~/Desktop]
$ logger -n 127.0.0.1 -P 514 -d "Health Check Successful"
(rania@kali) - [~/Desktop]
$ sudo docker exec -it logger tail -5 /var/log/messages
Jul 17 17:44:04 31926ec413ed syslog-ng[1]: Log statistics: processed='source(s_network)=0', connections='src.syslog(s_network.afsocket_sd(stream,AF_INET(0.0.0.0:6514)))=0', processed='global(msg_clones)=0', dropped='global(internal_source)=0', processed='global(internal_source)=1', queued='global(s_network.afsocket_sd(stream,AF_INET(0.0.0.0:601)))=0', queued='global(scratch_buffers_count)=0', processed='center(queued)=2', connections='src.network(s_network.afsocket_sd(stream,AF_INET(0.0.0.0:514)))=0', processed='src.internal(s_local0)=1', stamp='src.internal(s_local0)=1784305468', processed='source(s_local)=1', processed='global(sdata_updates)=0', processed='global(payload_realtime)=0', processed='center(received)=1', queued='global(scratch_buffers_bytes)=0'
Jul 17 17:54:04 31926ec413ed syslog-ng[1]: Log statistics: processed='source(s_network)=0', connections='src.syslog(s_network.afsocket_sd(stream,AF_INET(0.0.0.0:6514)))=0', processed='global(msg_clones)=0', dropped='global(internal_source)=0', processed='global(internal_source)=2', queued='global(internal_source)=0', processed='destination(d_local)=0', connections='src.syslog(s_network.afsocket_sd(stream,AF_INET(0.0.0.0:601)))=0', queued='global(scratch_buffers_count)=0', processed='center(queued)=4', connections='src.network(s_network.afsocket_sd(stream,AF_INET(0.0.0.0:514)))=0', processed='src.internal(s_local0)=2', stamp='src.internal(s_local0)=1784310244', processed='source(s_local)=2', processed='global(sdata_updates)=0', processed='global(payload_realtime)=2', processed='center(received)=2', queued='global(scratch_buffers_bytes)=0'
Jul 17 17:57:22 kali rania: Health test Fri Jul 17 01:57:22 PM EDT 2026
Jul 17 22:59:42 kali rania: TEST222
Jul 17 14:00:53 kali rania: Health Check Successful

```

Syslog capture (logger "test" → view in container logs)

Task 3: Python Automation

set_ip.py: takes --ip, --gw, --iface; applies via Netplan; logs to /var/log/ip-config.log.

```

Session Actions Edit View Help

(rania@kali)-[~/Desktop/Python_Scripts]
$ sudo python3 set_ip.py --ip 192.168.56.20/24 --gw 192.168.56.1 --iface eth0

** (configure:14545): WARNING **: 15:04:08.448: Permissions for /etc/netplan/99-set-ip-eth0.yaml are too open. Netplan configuration should NOT be accessible by others.

** (configure:14545): WARNING **: 15:04:08.450: 'gateway4' has been deprecated, use default routes instead.
See the 'Default routes' section of the documentation for more details.

** (process:14538): WARNING **: 15:04:08.716: Permissions for /etc/netplan/99-set-ip-eth0.yaml are too open. Netplan configuration should NOT be accessible by others.

** (process:14538): WARNING **: 15:04:08.717: 'gateway4' has been deprecated, use default routes instead.
See the 'Default routes' section of the documentation for more details.

** (process:14538): WARNING **: 15:04:09.070: Permissions for /etc/netplan/99-set-ip-eth0.yaml are too open. Netplan configuration should NOT be accessible by others.

** (process:14538): WARNING **: 15:04:09.070: 'gateway4' has been deprecated, use default routes instead.
See the 'Default routes' section of the documentation for more details.
systemd-networkd is not running, output might be incomplete.
Failed to reload network settings: Unit dbus-org.freedesktop.network1.service not found.
Falling back to a hard restart of systemd-networkd.service
Applied IP 192.168.56.20/24 on eth0 with gateway 192.168.56.1

(rania@kali)-[~/Desktop/Python_Scripts]
$ cat /var/log/ip-config.log
[2026-07-18 14:41:41] ERROR applying IP config: [Errno 2] No such file or directory: '/etc/netplan/99-set-ip-eth0.yaml'
[2026-07-18 15:04:10] Applied IP 192.168.56.20/24 on eth0 with gateway 192.168.56.1

```

(set_ip.py running)

Explanation: The script automates the manual task which was being done in Task 1.

Real-world scenario: When you have to apply static ip addresses over a large number of machines, you will not prefer to do it by hand. You can just run the script and get the static ip addresses

monitor.py

1. Ping 192.168.56.10 (Nginx host)
2. HTTP-check Nginx: GET / to http://192.168.56.10:8080 (use requests lib)
3. Syslog test: run logger "Health test \$(date)" → ensure message appears in Syslog-NG container (docker logs logger)
4. Log status lines to healthcheck.log with timestamps

```

(rania@kali)-[~/Desktop/Python_Scripts]
$ python3 monitor.py
[2026-07-18 15:11:41] PING 192.168.56.10 OK | HTTP 200 OK | SYSLOG RECEIVED

(rania@kali)-[~/Desktop/Python_Scripts]
$ cat healthcheck.log
[2026-07-18 15:09:30] PING 192.168.56.10 OK | HTTP FAIL | SYSLOG RECEIVED
[2026-07-18 15:09:30] ALERT: One or more checks failed!
[2026-07-18 15:11:41] PING 192.168.56.10 OK | HTTP 200 OK | SYSLOG RECEIVED

(rania@kali)-[~/Desktop/Python_Scripts]
$

```

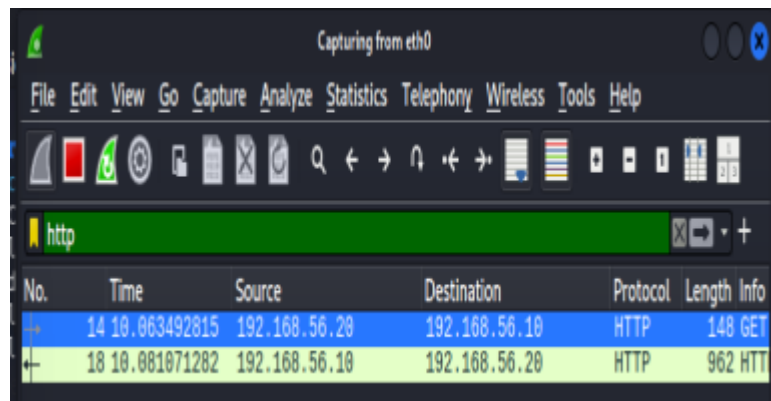
(monitor.py running)

Explanation: The script checks whether the infra-server can be pinged, can we get the web server response and whether logging is working

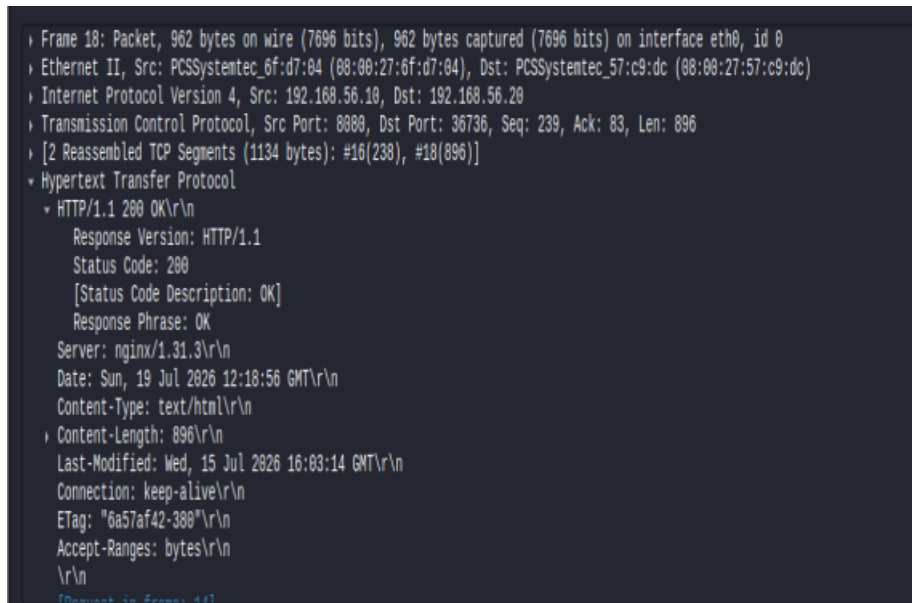
Real-world scenario: This basically resembles a tiny monitoring tool

Bonus: Packet Analysis Section

1. HTTP traffic (admin-tools → Nginx) – screenshot + header analysis



(Packet capture on wirshark shows http traffic)



(http GET)

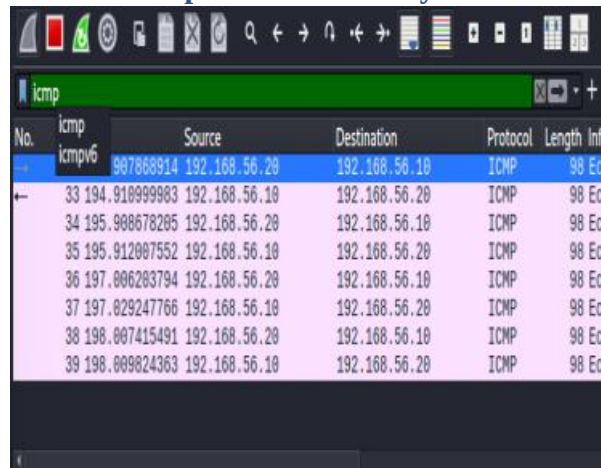

```

Frame 14: Packet, 148 bytes on wire (1184 bits), 148 bytes captured (1184 bits) on interface eth0, id 0
Ethernet II, Src: PCSSystemtec_57:c9:dc (08:00:27:57:c9:dc), Dst: PCSSystemtec_6f:d7:04 (08:00:27:6f:d7:04)
Internet Protocol Version 4, Src: 192.168.56.20, Dst: 192.168.56.10
Transmission Control Protocol, Src Port: 36736, Dst Port: 8080, Seq: 1, Ack: 1, Len: 82
Hypertext Transfer Protocol
GET / HTTP/1.1\r\n
Host: 192.168.56.10:8080\r\n
User-Agent: curl/8.17.0\r\n
Accept: */*\r\n
\r\n
[Response in frame: 18]
[Full request URI: http://192.168.56.10:8080/]

```

(http 200OK)

2. ICMP echo between VMs – prove connectivity & TTL values



No.	icmp	Source	Destination	Protocol	Length	Info
32	icmpv6	907868914	192.168.56.20	ICMP	98	Ech
33	194.910999903	192.168.56.10	192.168.56.20	ICMP	98	Ech
34	195.908678205	192.168.56.20	192.168.56.10	ICMP	98	Ech
35	195.912007552	192.168.56.10	192.168.56.20	ICMP	98	Ech
36	197.006283794	192.168.56.20	192.168.56.10	ICMP	98	Ech
37	197.029247766	192.168.56.10	192.168.56.20	ICMP	98	Ech
38	198.007415491	192.168.56.20	192.168.56.10	ICMP	98	Ech
39	198.009024363	192.168.56.10	192.168.56.20	ICMP	98	Ech

(Packet capture on wireshark shows icmp traffic)

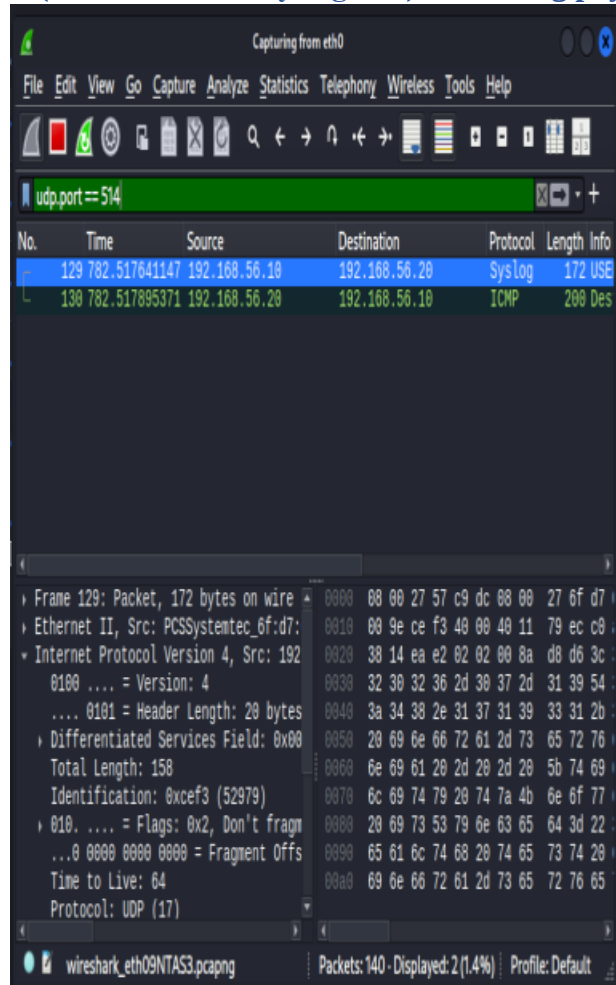
No.	Time	Source	Destination	Protocol	Length	Info
32	194.907868914	192.168.56.20	192.168.56.10	ICMP	98	Ech
33	194.910999983	192.168.56.10	192.168.56.20	ICMP	98	Ech
34	195.908678205	192.168.56.20	192.168.56.10	ICMP	98	Ech
35	195.912007552	192.168.56.10	192.168.56.20	ICMP	98	Ech
36	197.006203794	192.168.56.20	192.168.56.10	ICMP	98	Ech
37	197.029247766	192.168.56.10	192.168.56.20	ICMP	98	Ech
38	198.007415491	192.168.56.20	192.168.56.10	ICMP	98	Ech
39	198.009824363	192.168.56.10	192.168.56.20	ICMP	98	Ech

Frame 32: Packet, 98 bytes on wire (7...)	0000	00 00 27 6f d7 04 08 00	27 57 c9
Ethernet II, Src: PCSSystemtec_57:c9:	0010	00 54 82 b4 40 00 40 81	c6 85 c0
Internet Protocol Version 4, Src: 192	0020	38 0a 08 00 12 ad 00 02	00 01 08
0100 = Version: 4	0030	00 00 59 51 00 00 00 00	00 00 10
.... 0101 = Header Length: 20 bytes	0040	16 17 18 19 1a 1b 1c 1d	1e 1f 20
Differentiated Services Field: 0x00	0050	26 27 28 29 2a 2b 2c 2d	2e 2f 30
Total Length: 84	0060	36 37	
Identification: 0x82b4 (33460)			
010. = Flags: 0x2, Don't fragm			
...0 0000 0000 0000 = Fragment Offs			
Time to Live: 64			
Protocol: ICMP (1)			

Internet Control Message Protocol: Protocol | Packets: 41 - Displayed: 8 (19.5%) | Profile: Default

(ICMP packet detail showing Time To Live (TTL) = 64, confirming direct connectivity)

3. Syslog UDP 514 (infra-server → Syslog-NG) – show log payload

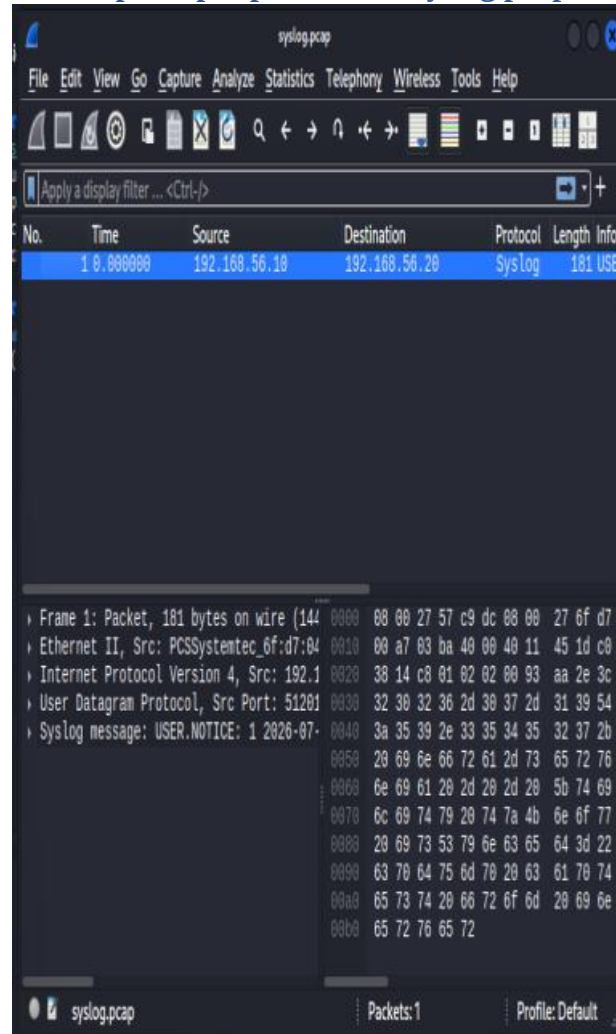


(Wireshark capture filtered on `udp.port == 514`, showing the syslog payload sent from infra-server to admin-tools)

4. Security note: plaintext HTTP vs HTTPS

For the working of this lab all the traffic of this lab is sent over http. http traffic is not encrypted so all the headers even can be viewed in plain-text. Now if the same was to be done using https the traffic would be encrypted using TLS. In a real-world scenario traffic is not communicated using http but rather with https

5. Extra: capture with tcpdump -i port 514 -w syslog.pcap and open in Wireshark



Reflection: What I Learned This Week

1. This week I learnt that when we are sending traffic over one host-only adapter then we need to up another adapter on NAT to access the internet
2. A gateway ip was posing default gateway conflicts during the whole work on kali and that had to be removed manually again and again